MONITOR Z · L01.3

Clase 3 — Entrar y salir: prólogo, save area, epílogo y RC

Cuaderno de estudio · versión del 2026-10-28

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01.3 — Clase 3 · Entrar y salir: prólogo, save area, epílogo y RC

Temario de la L01: 1. La forma de un programa · 2. Los registros y las instrucciones de todos los días · **3. Entrar y salir: prólogo, save area, epílogo y RC** · 4. Los datos: DS, DC, literales, el PARM y el `EX` · 5. Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE/RMODE.

Objetivo de esta clase: entender los 30 bytes con los que ZTKHOLA entra y los 18 con los que sale. Al terminar tenés que poder:

- dibujar una save area y decir qué va en cada palabra;
- explicar cada renglón del prólogo y del epílogo, y por qué van en ese orden;
- seguir el RC desde R15 hasta el `COND CODE` del JESYSMSG y el `RC=` del `$HASP395`.

> Fuentes: - `monitor_z/asm/ZTKHOLA.asm`, renglones 12–20 y 72–78. - El listado de **JOB24962**, sentencias 12–20 y 100–106, y la *General Purpose Register Cross Reference* (página 10). - JESJCL, JESYSMSG y JESMSGLG de JOB24962: los `COND=`, los `IEF142I … COND CODE` y el `$HASP395 … RC=0008`. - El listado de **JOB34775** (ZTKMON v0.9): sentencias 10126–10137 y 10329–10334. - *z/OS MVS Assembler Services Guide* (SA23-1368), capítulo «Linkage conventions».

Respuestas de la clase 2

1. `58D0 D004` es `L 13,4(,13)`: `58` = L, `D` = carga R13, `0` = sin índice, `D` = base R13, `004` = desplazamiento 4. En la palabra 4 de **tu** save area quedó la dirección de la save area **del que te llamó** (la puso `ST 13,4(,2)`). Es lo primero que hay que recuperar para salir: sin eso no sabés dónde están guardados sus registros. Es el tema de hoy.
2. Con `L 2,SAVEAREA`, R2 tendría los primeros 4 bytes **de adentro** de la save area: un `DS`, sin valor puesto. El `ST 13,4(,2)` del renglón 17 escribiría en una dirección cualquiera. Qué pasaría después **no está medido**: depende de qué hubiera en esos bytes.
3. Con `PARM='A'`: R3 = 1, `LTR` deja CC 2 y `BZ` no salta. `CH` contra 100 deja CC 1, así que `BNH` **salta**. `BCTR` deja R3 = 0, y el `EX` ejecuta el `MVC` con

largo guardado 0, que copia **1 byte**: la `A`. RC 4. Es lo que se espera leyendo los bytes; esa corrida **no se hizo**.

1. El contrato: z/OS te llama como a una subrutina

Tu programa no «arranca»: **lo llaman**. Cuando el paso `RUN1` ejecuta `PGM=ZTKHOLA`, z/OS carga el módulo y le salta igual que una rutina llama a otra. Y en el momento del salto deja cuatro registros preparados:

Registro	Qué trae al entrar
R13	la dirección de una **save area** de 72 bytes, del que te llamó, para que guardes ahí sus registros
R14	la dirección a la que tenés que **volver**
R15	tu propia dirección de entrada
R1	la dirección de la lista de parámetros (el PARM, clase 4)

Analogía: te prestan una **oficina** para trabajar. Antes de mover nada, sacás una **foto** de cómo estaba el escritorio (guardás los registros). Dejás anotado **de quién es** la oficina y dónde está tu propia libreta (encadenás las save areas). Al irte, ponés todo como en la foto, dejás una **nota** en la puerta (el RC en R15) y salís por donde entraste (`BR 14`).

2. La save area: 18 palabras, siempre iguales

Una save area mide **72 bytes**: 18 fullwords. Por eso tu renglón 85 dice `SAVEAREA DS 18F`, y el `Loc` avanzó de `0000D4` a `00011C` (X'48' = 72, la pregunta 2 de la clase 1).

Desplazamiento	Palabra	Qué guarda
+0	1	(para lenguajes de alto nivel; ZTKHOLA no lo usa)
+4	2	**HSA** (*high save area*): la dirección de la save area **del que me llamó**
+8	3	**LSA** (*low save area*): la dirección de la save area **del que yo llame**
+12	4	R14
+16	5	R15

+20 … +68	6–18	R0, R1, R2 … R12 (13 registros, 52 bytes)

> 💡 **La regla de oro:** los registros se guardan en la save area **del que te llamó**, no en la tuya. La tuya es para el que **vos** llames: `OPEN`, `PUT`, `CLOSE`.

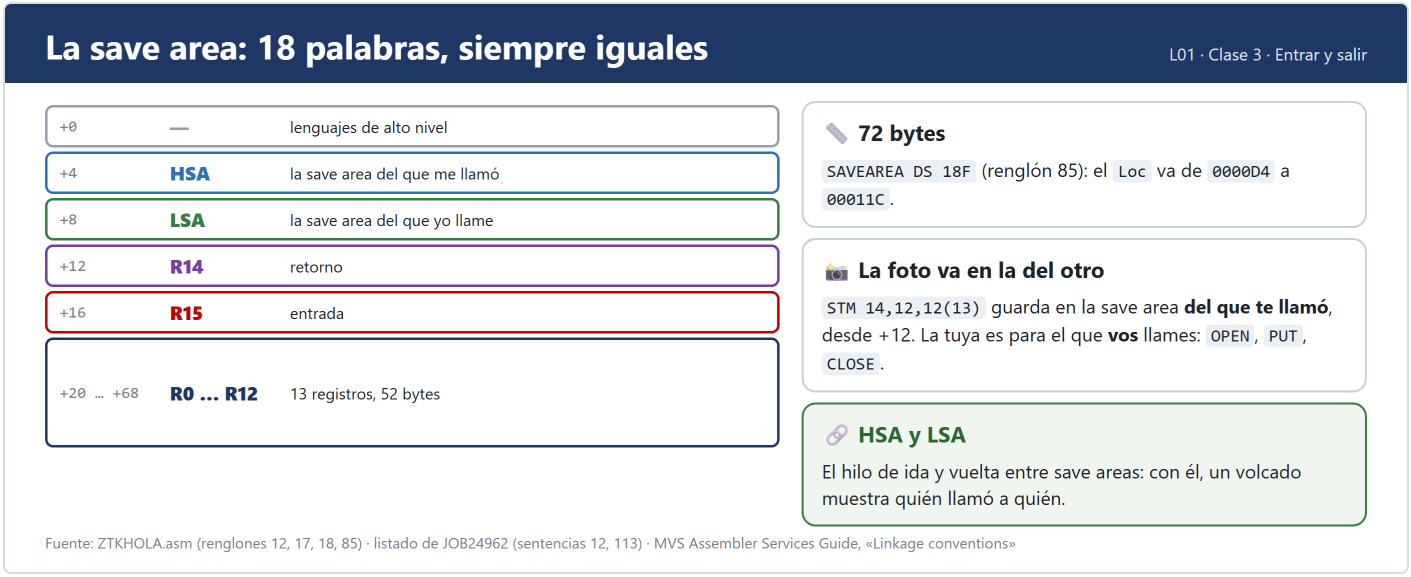


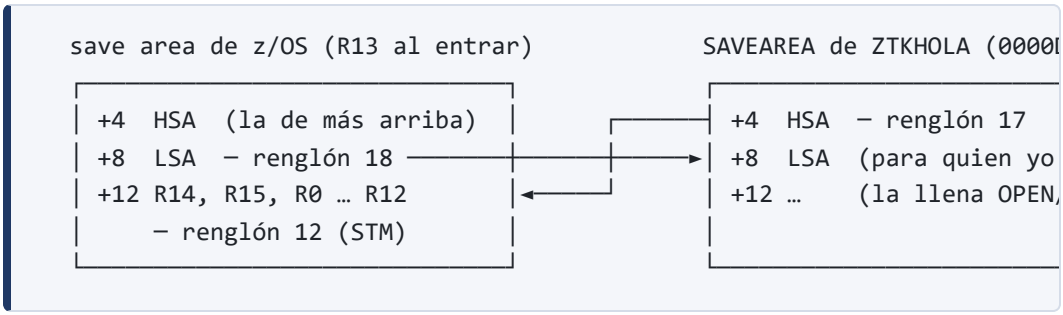
Lámina L01-18_la_save_area

3. El prólogo, renglón por renglón (30 bytes)

Renglón	Loc	Bytes	Sentencia	Qué hace y por qué ahí
12	`000000`	`90EC D00C`	`STM 14,12,12(13)`	La foto. R14, R15, R0 … R12 van a la save area del que llamó, a partir de +12. Va **primero**, antes de tocar ningún registro.
13	`000004`	`18CF`	`LR 12,15`	R15 trae tu dirección de entrada: la copiás en R12 para que sea tu base.
14	—	—	`USING ZTKHOLA,12`	La promesa al ensamblador (clase 2). No genera bytes.
15	`000006`	`5010 C0CC`	`ST 1,SALVAR1`	Guardás R1, la lista del PARM, antes de que `OPEN` lo pise (lo hace en la sent. 30: `BAS 1,*+8`).
16	`00000A`	`4120 C0D4`	`LA 2,SAVEAREA`	R2 = la **dirección** de tu save area.

17	`00000E`	`50D0 2004`	`ST 13,4(,2)`	**Mi HSA** = la del que me llamó. El hilo para volver.
18	`000012`	`5020 D008`	`ST 2,8(,13)`	**Su LSA** = la mía. El hilo hacia adelante.
19	`000016`	`18D2`	`LR 13,2`	Desde ahora, R13 apunta a **tu** save area: lista para el que vos llames.
20	`000018`	`D203 C0D0 C290`	`MVC RCFINAL,=F'8'`	El RC por omisión: si nada cambia, sale con 8 (sin PARM).

El encadenamiento, dibujado:



Con este hilo de ida y vuelta, cualquiera que mire un volcado puede recorrer la cadena de llamadas: quién llamó a quién.

> ⚠️ **El orden importa.** Si `LR 13,2` fuera antes que `ST 13,4(,2)`, se perdería la dirección de la save area del que llamó: R13 ya no la tendría, y no habría forma de volver.

El prólogo: 30 bytes para entrar sin romper nada

L01 · Clase 3 · Entrar y salir

Renglón	Loc	Bytes	Sentencia	Qué hace
12	`000000`	`90EC D00C`	`STM 14,12,12(13)`	la foto, **antes** de tocar nada
13	`000004`	`18CF`	`LR 12,15`	R12 = mi dirección de entrada
14	—	—	`USING ZTKHOLA,12`	la promesa (sin bytes)
15	`000006`	`5010 C0CC`	`ST 1,SALVAR1`	salvo R1 (el PARM) antes del `OPEN`
16	`00000A`	`4120 C0D4`	`LA 2,SAVEAREA`	R2 = dirección de mi save area
17	`00000E`	`50D0 2004`	`ST 13,4(,2)`	**mi HSA** = la del que me llamó
18	`000012`	`5020 D008`	`ST 2,8(,13)`	**su LSA** = la mía
19	`000016`	`18D2`	`LR 13,2`	R13 = mi save area
20	`000018`	`D203 C0D0 C290`	`MVC RCFINAL,=F'8'`	RC por omisión: 8

⚠️ **El orden importa.** Si `LR 13,2` fuera antes que `ST 13,4(,2)`, se perdería la save area del que llamó y no habría forma de volver.

Fuente: listado de JOB24962 (sentencias 12–20)

Lámina L01-19_el_prologo

4. El epílogo, renglón por renglón (18 bytes)

Renglón	Loc	Bytes	Sentencia	Qué hace y por qué ahí
73	`0000BA`	`58F0 C0D0`	`L 15,RCFINAL`	El RC pasa de memoria a R15 **recién ahora**: `PUT` y `CLOSE` pisan R15 (clase 2).
75	`0000BE`	`58D0 D004`	`L 13,4(,13)`	Seguís el hilo: R13 vuelve a ser la save area del que llamó.
76	`0000C2`	`58E0 D00C`	`L 14,12(,13)`	La dirección de retorno, de la foto.
77	`0000C6`	`980C D014`	`LM 0,12,20(13)`	R0 … R12 como estaban. **R15 no**, porque lleva el RC.
78	`0000CA`	`07FE`	`BR 14`	Volvés a z/OS.

Por qué `LM 0,12` y no `LM 14,12`: `LM 14,12,12(13)` sería la vuelta exacta del `STM` del prólogo, pero también repondría **R15** con lo que tenía al entrar (tu dirección de entrada), y el RC se perdería. Por eso se carga R14 sola y el `LM` arranca en R0.

La *General Purpose Register Cross Reference* (página 10 del listado) confirma el contrato. **R4 a R11** aparecen sólo en las sentencias 12 (el `STM`) y 105 (el `LM`):

ZTKHOLA no los usa, sólo los guarda y los repone. **R15** se modifica en 81–82 (el `PUT`) y en 101 (el RC).

El epílogo: 18 bytes para salir dejando todo como estaba

L01 · Clase 3 · Entrar y salir

Renglón	Loc	Bytes	Sentencia	Qué hace
73	`0000BA`	`58F0 C0D0`	`L 15,RCFINAL`	el RC a R15 **recién ahora**
75	`0000BE`	`58D0 D004`	`L 13,4(,13)`	sigo el hilo: R13 = la del que llamó
76	`0000C2`	`58E0 D00C`	`L 14,12(,13)`	la dirección de retorno
77	`0000C6`	`980C D014`	`LM 0,12,20(13)`	R0 … R12 como estaban; **R15 no**
78	`0000CA`	`07FE`	`BR 14`	vuelvo a z/OS

❌ **¿Por qué no `LM 14,12,12(13)`?**

Sería la vuelta exacta del `STM`, pero repondría **R15** con la dirección de entrada y el RC se perdería.

Register Cross Reference

R4 a R11 aparecen sólo en las sentencias 12 y 105: ZTKHOLA no los usa, sólo los guarda y los repone.

ZTKMON, idéntico

Sentencias 10331–10334: `58D0 D004` · `58E0 D00C` · `980C D014` · `07FE`. Los mismos bytes.

Fuente: listado de JOB24962 (sentencias 100–106; página 10) · listado de JOB34775 (sentencias 10329–10334)

Lámina L01-20_el_epilogo

5. El RC: de R15 al `$HASP395`

Lo que dejás en R15 al hacer `BR 14` es lo que z/OS anota como **código de terminación del paso**. ZTKHOLA tiene un contrato de tres valores, que está en el comentario de la cabecera (renglones 6–8):

RC	Cuándo	¿Medido?
4	vino PARM y se escribió la línea	✅ `IEF142I ZTKHOLA RUN1 - STEP WAS EXECUTED - COND CODE 0004`
8	no vino PARM	✅ `IEF142I ZTKHOLA RUN2 - STEP WAS EXECUTED - COND CODE 0008`
12	no se pudo abrir SYSPRINT	❌ **no medido**: ninguna corrida falló el `OPEN`

Tres lugares donde se lee el RC:

Dónde	Qué dice en JOB24962
JESYSMSG	un `IEF142I … COND CODE` por paso: 0000, 0000, 0004, 0008
JESMSGLG	la tabla de pasos: `ASM 00`, `LKED 00`, `RUN1 04`, `RUN2 08`

`$HASP395`	`ZTKHOLA ENDED - RC=0008` : uno solo, para todo el job

El `RC=` del `$HASP395` es, por omisión, el **más alto** de los pasos (`JOBRC=MAXRC`). En este job el más alto y el último coinciden (RUN2, 08), así que JOB24962 no sirve para distinguirlos.

El RC también decide qué pasos corren. El paso `LKED` tiene `COND=(4,LT,ASM)` : «**no** corras este paso si 4 es menor que el RC de ASM». Con el ensamblado en 00 corrió. Si ASM hubiera dado 8, LKED, RUN1 y RUN2 se habrían salteado, porque los tres tienen esa condición.



Lámina L01-21_el_rc

6. Lo mismo en ZTKMON

El prólogo de ZTKMON (sentencias 10126–10137) es **el mismo** que el de ZTKHOLA. Sólo agrega las dos bases R11 y R10 de la clase 2. El epílogo (10329–10334) también:

```
00028A 58F0 BE38      01E38 10329          L     R15,RCFINAL
00028E 58D0 D004      00004 10331          L     R13,4(,R13)
000292 58E0 D00C      0000C 10332          L     R14,12(,R13)
000296 980C D014      00014 10333          LM    R0,R12,20(R13)
00029A 07FE                 10334          BR    R14
```

Los mismos bytes que tu epílogo (`58D0 D004` , `58E0 D00C` , `980C D014` , `07FE`). Cambia sólo el `L R15,RCFINAL` , porque en ZTKMON `RCFINAL` está más lejos

(base R11).

Lo que ZTKMON tiene y ZTKHOLA no son **rutinas internas**. Se llaman con `BAL R14,rutina`, guardan R14 en una palabra propia (por ejemplo `GUARDAHO`) y vuelven con `L R14,GUARDAHO` + `BR R14`, sin save area: un enlace más liviano, sólo dentro del programa. En el listado de JOB34775 hay **60** `BR R14`.

Resumen

Idea	En una línea	Testigo
El contrato	al entrar: R13 = save area, R14 = retorno, R15 = entrada, R1 = PARM	renglones 12–15
Save area	72 bytes: +4 HSA, +8 LSA, +12 R14, +16 R15, +20 R0 … R12	`SAVEAREA DS 18F`, 0D4 → 11C
Prólogo	foto (`STM`), base, guardar R1, encadenar ida y vuelta, R13 = la mía	30 bytes, `000000`–`00001D`
Epílogo	RC a R15, seguir el hilo, reponer R14 y R0 … R12, `BR 14`	18 bytes, `0000BA`–`0000CB`
`LM 0,12`	no `LM 14,12`: R15 lleva el RC	sentencia 105
El RC	R15 → `COND CODE` del paso → `RC=` del job (el más alto)	`IEF142I` 0004 y 0008, `$HASP395 RC=0008`
ZTKMON	mismo prólogo y epílogo, byte por byte; rutinas internas con `BAL R14`	JOB34775, sentencias 10126–10334

Preguntas de comprobación

1. El prólogo guarda 15 registros con `STM 14,12,12(13)`, pero el epílogo no hace `LM 14,12,12(13)`. Si alguien lo «arreglara» para que quedara simétrico, ¿con qué RC terminaría RUN1? ¿Por qué?
2. Imaginá que te olvidás el renglón 17 (`ST 13,4(,2)`). El programa ensambla sin errores. ¿Qué carga el `L 13,4(,13)` del epílogo? ¿Qué pasa en los dos renglones siguientes?
3. Si el paso ASM hubiera terminado con RC 8, ¿qué pasos habrían corrido? ¿Qué diría el `$HASP395`?

Mis respuestas

Relacionado: *L01.2 - Clase 2 - Los registros y las instrucciones de todos los dias* · *L01.4 - Clase 4 - Los datos - DS, DC, literales, el PARM y el EX* · *P01 - ZTKHOLA* · *Ruta de aprendizaje*.

Notas